

Computer Networks

Module 1

Networking Foundations

- *Layering & Protocols • Topology • Encapsulation • Multiplexing • OSI • TCP/IP • Physical Layer • Internet Architecture • API*
- *Performance Metrics • High Speed Networks*

Reference: Larry L. Peterson, Bruce S. Davie, "Computer Networks : A Systems Approach", The Morgan Kaufmann Series in Networking, 6th Edition, 2021

Module 1 - Topics

S.No	Topic
1	Introduction , Layering & Protocols
2	Network Topology
3	Encapsulation
4	Multiplexing & Demultiplexing
5	OSI Reference Model
6	TCP/IP Model
7	Basics of the Physical Layer
8	Internet Architecture
9	Application Programming Interface (Sockets)
10	Performance Metrics & High-Speed Networks.

Learning Objectives

- Explain why abstraction and layering are essential to managing the complexity of network design.
- Distinguish between a protocol's service interface and its peer interface, and trace a message through a protocol graph.
- Classify network topologies and evaluate their cost, reliability, and scalability trade-offs.
- Describe encapsulation and de-encapsulation, and identify headers, trailers, and payloads at each layer.
- Explain multiplexing/demultiplexing and the role of a demux key in delivering data to the correct process.
- Compare the OSI 7-layer reference model with the 4-layer TCP/IP (Internet) architecture.
- Identify core physical layer concepts: transmission media, signals, bandwidth, and bit rate.
- Describe the Internet's hourglass architecture and explain why IP occupies the narrow waist.

Why We Need a Network Architecture

- A network must provide connectivity that is general, cost-effective, fair, and robust among a large number of computers.
- Networks do not stay fixed — they must evolve as underlying technologies change and as new applications place new demands on them.
- Networks must be manageable by humans with varying levels of skill.
- Designing a network that satisfies all these requirements at once is not a small task.
- **Network designers respond with general blueprints called network architectures that guide design and implementation.**

Blueprint Analogy

Just as a building architect provides a blueprint that guides electricians, plumbers, and carpenters independently, a network architecture provides a blueprint so hardware and software from many vendors can interoperate.

Abstraction — The Core Design Tool

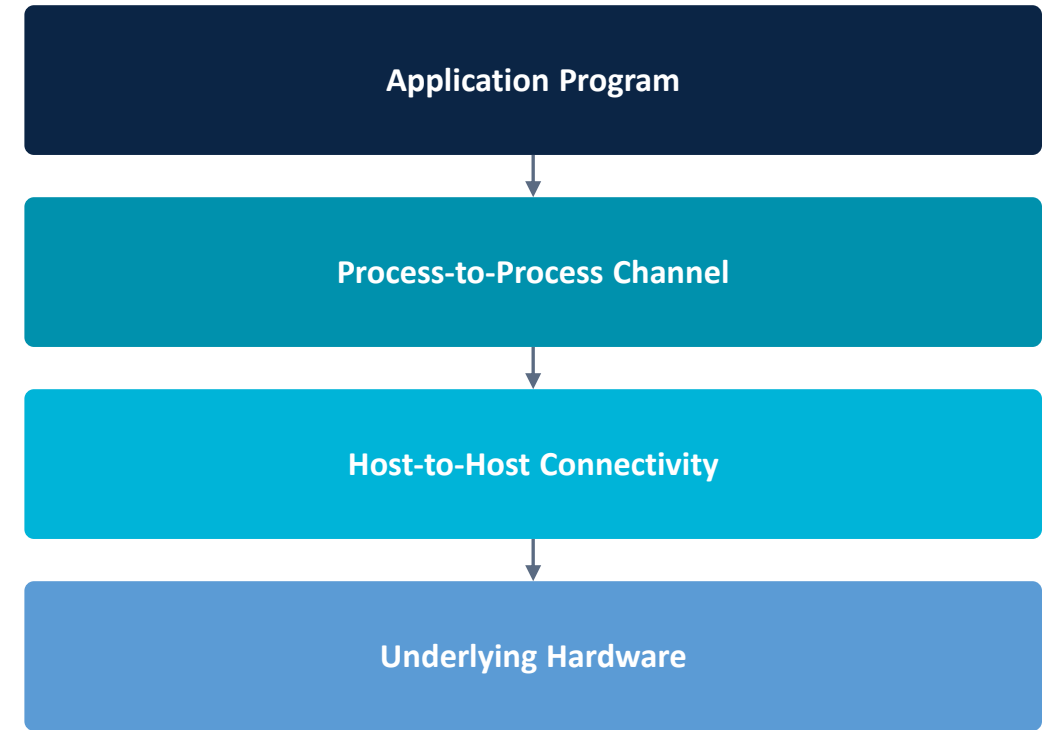
- **Abstraction: hiding implementation details behind a well-defined interface.**
- System designers use abstraction as the fundamental tool to manage complexity.
- Steps in creating a useful abstraction:
 - Define a model that captures an important aspect of the system.
 - Encapsulate the model in an object with a manipulable interface.
 - Hide implementation details from the object's users.
- Example: a communication channel is an abstraction that hides network complexity from application writers.

Key Takeaway

Abstraction in computer networks means providing a simple interface for communication while hiding the complex details of data transmission, routing, error handling, and protocol operations. This allows applications to communicate easily without needing to understand how the underlying network works.

What Is Layering?

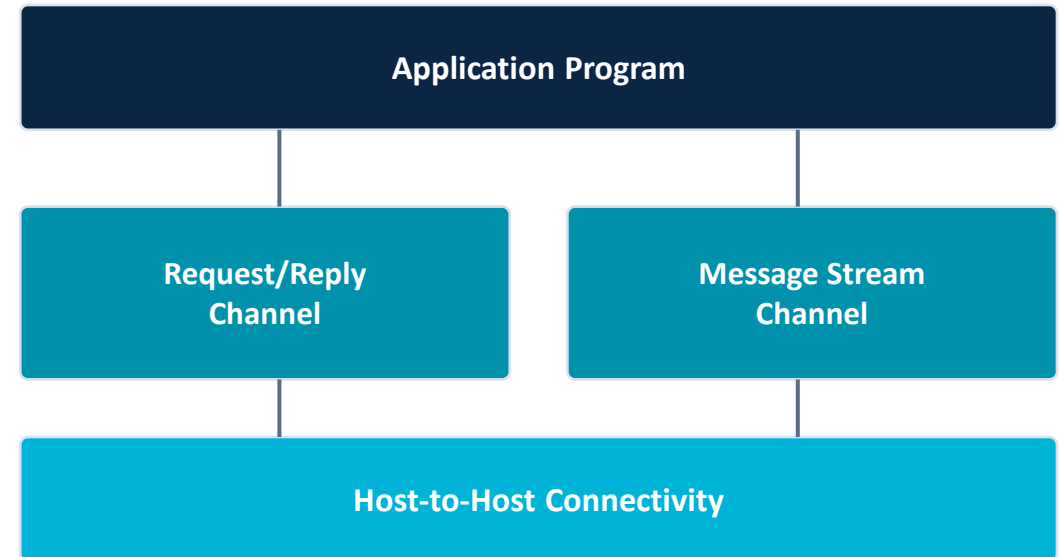
- Start with services offered by the underlying hardware.
- Add a sequence of layers, each providing a higher, more abstract level of service.
- A high layer is implemented in terms of the services of the layer below it.
- Layering gives two big benefits:
 - Decomposition — break one hard problem into several manageable ones.
 - Modularity — change one layer's implementation without touching the rest.



Each layer builds on the service below it and hides that layer's complexity from the one above.

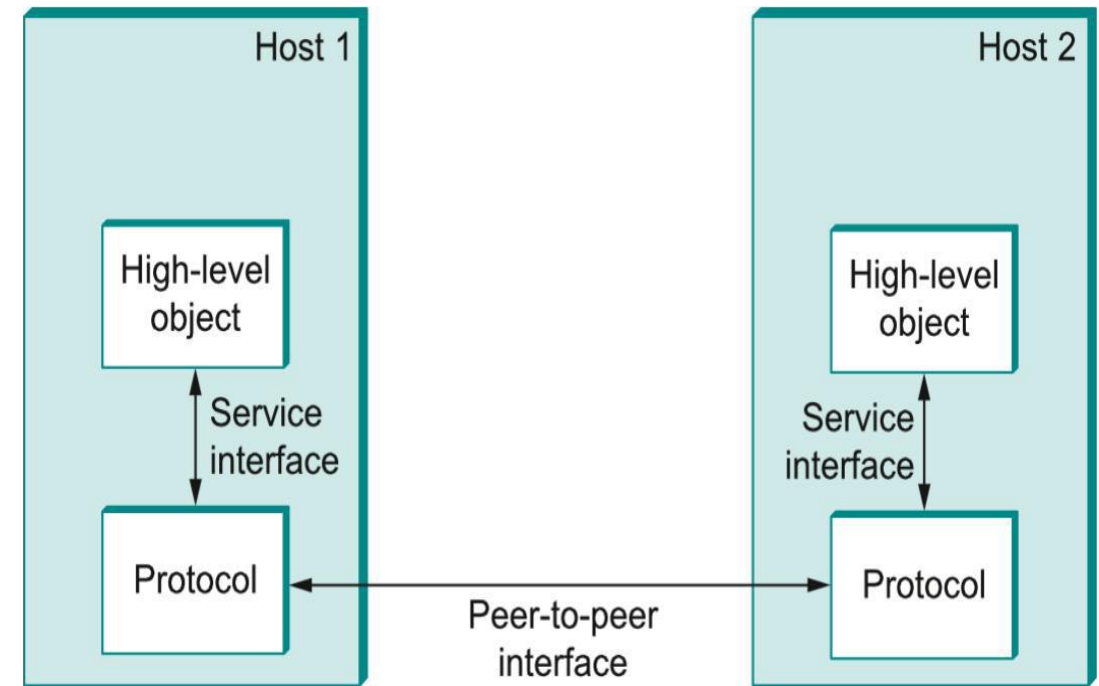
Multiple Abstractions at One Layer

- Layering is not always a strictly linear stack.
- A given level can offer several alternative abstractions, each built on the same lower-level service.
- Example: a request/reply channel and a message-stream channel can both sit at the same level, offering different services to the layer above.
- The application picks whichever channel type best fits what it needs to do.



Protocols: Two Interfaces

- **Protocol:** An object that provides a communication service used by higher-level objects to exchange messages.
- Every protocol defines two distinct interfaces:
 - Service interface — operations local objects use to invoke the protocol's service (e.g. send / receive).
 - Peer interface — the form and meaning of messages exchanged with its counterpart on another machine.
- Example: HTTP's service interface lets a browser 'fetch a page'; its peer interface defines exactly how a GET request is formatted.

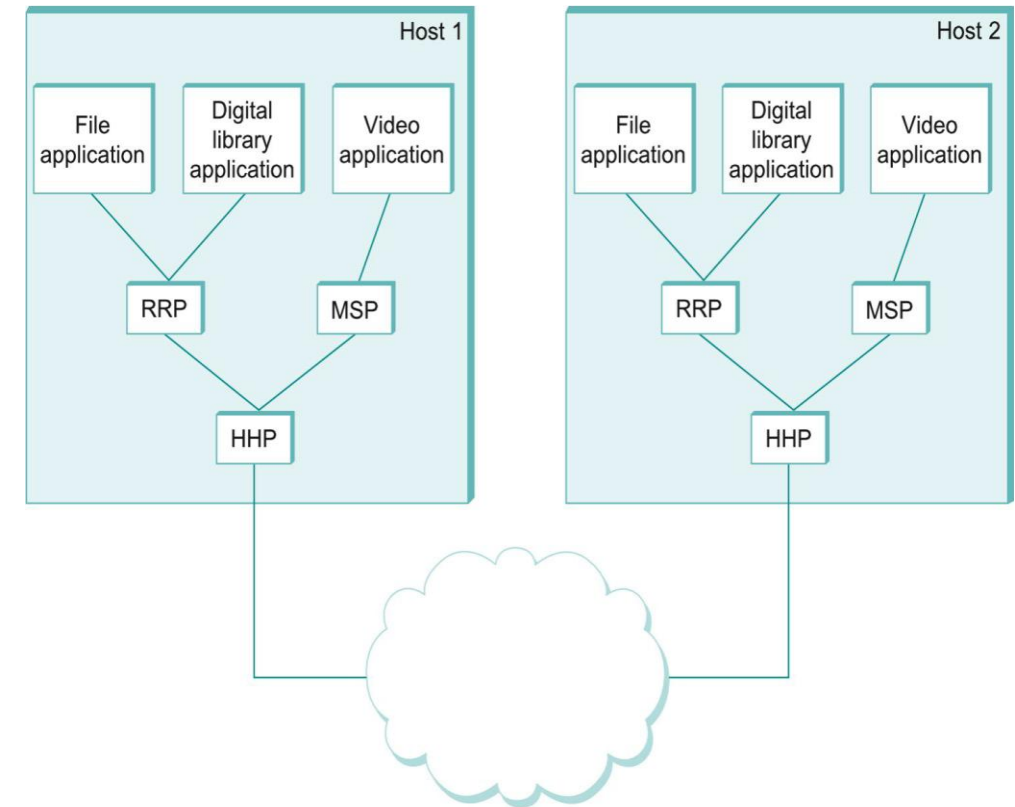


Service interfaces and peer interfaces

To summarize, a protocol defines a communication service that it exports locally (the service interface), along with a set of rules governing the messages that the protocol exchanges with its peer(s) to implement this service (the peer interface).

Protocol Graph: RRP, MSP, and HHP

- A protocol graph represents the suite of protocols in a system: nodes are protocols, edges show a 'depends on' relation.
- RRP (Request/Reply Protocol) and MSP (Message Stream Protocol) both depend on HHP (Host-to-Host Protocol).
- To send a message, the file application asks RRP; RRP invokes HHP; HHP delivers it to its peer on Host 2, which passes it back up to RRP, then to the application.
- This path — RRP over HHP — is called a protocol stack.



Specification vs. Implementation

- **Protocol specification:**

- A protocol specification is a document that defines how a protocol should work.
- Expressed using prose, pseudocode, state diagrams, and packet-format pictures.

- **Protocol implementation:**

- A module (software) that actually realises the specification.
- Different programmers can implement the same specification differently.

- **Interoperability:**

- Two or more implementations that correctly follow a specification and can successfully exchange messages are said to interoperate.

Standardization Bodies

IETF

Internet Engineering Task Force — defines the Internet architecture and specifies protocols such as TCP, UDP, IP, DNS, and BGP.

ISO

International Standards Organization — defined the OSI reference architecture and 7-layer model.

These standards specify how protocols should work and how they should be organized into layers. When companies implement these standards correctly, their devices can communicate seamlessly, regardless of who manufactured them.



Map the Postal System to a Protocol Stack

- ▶ In pairs, list the 'layers' involved when you post a letter: you writing a letter, an envelope with an address, the local post office, a transport truck/plane, and the receiving post office.
- ▶ For each layer, identify: (a) what service it offers the layer above, and (b) what 'peer' it communicates with on the receiving side.
- ▶ Identify one header (control information) that gets added at each stage — e.g., the envelope address is the 'header' added by the postal layer.
- ▶ Discuss: what does the letter writer NOT need to know about, thanks to abstraction, in order to send the letter successfully?
- ▶ Be ready to share your mapping and name which part corresponds to a service interface and which to a peer interface.



TOPIC 2

Network Topology

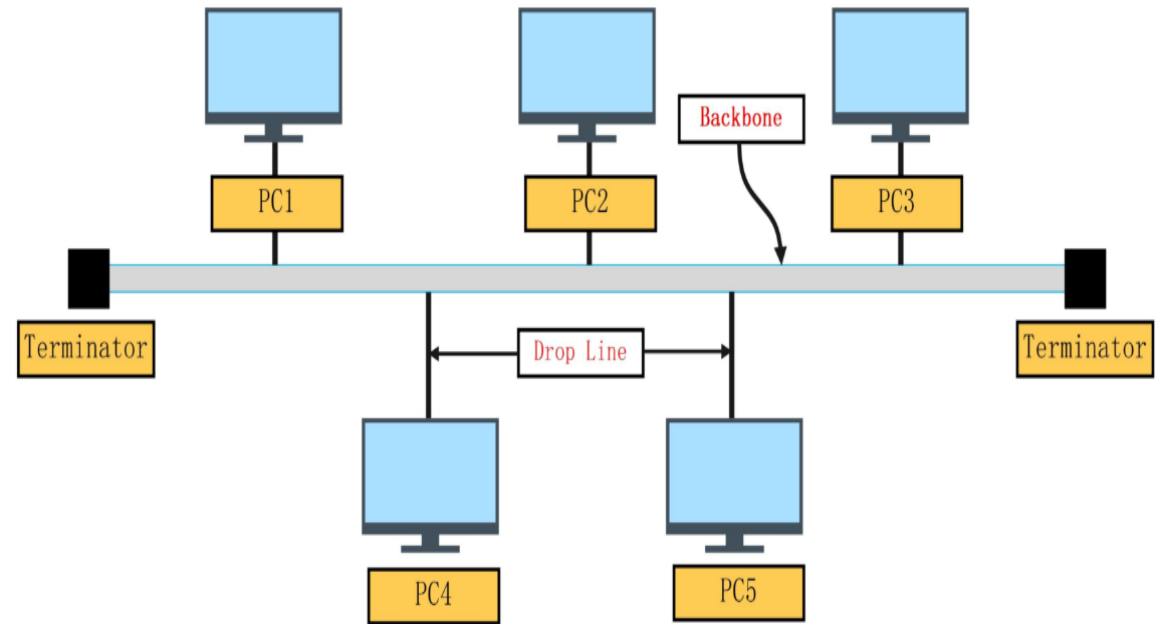
The physical and logical arrangement of nodes and links in a network.

What Is Network Topology?

- **Network topology:** the arrangement of nodes (computers, switches) and links (cables, wireless paths) that make up a network.
- Physical topology — the actual physical layout of cabling and devices.
- Logical topology — the path data actually takes as it flows through the network, which may differ from the physical layout.
- Topology choice affects: cost, cabling complexity, fault tolerance, ease of adding new nodes, and performance under load.
- Common topologies: Bus, Star, Ring, Mesh, Tree, and Hybrid.

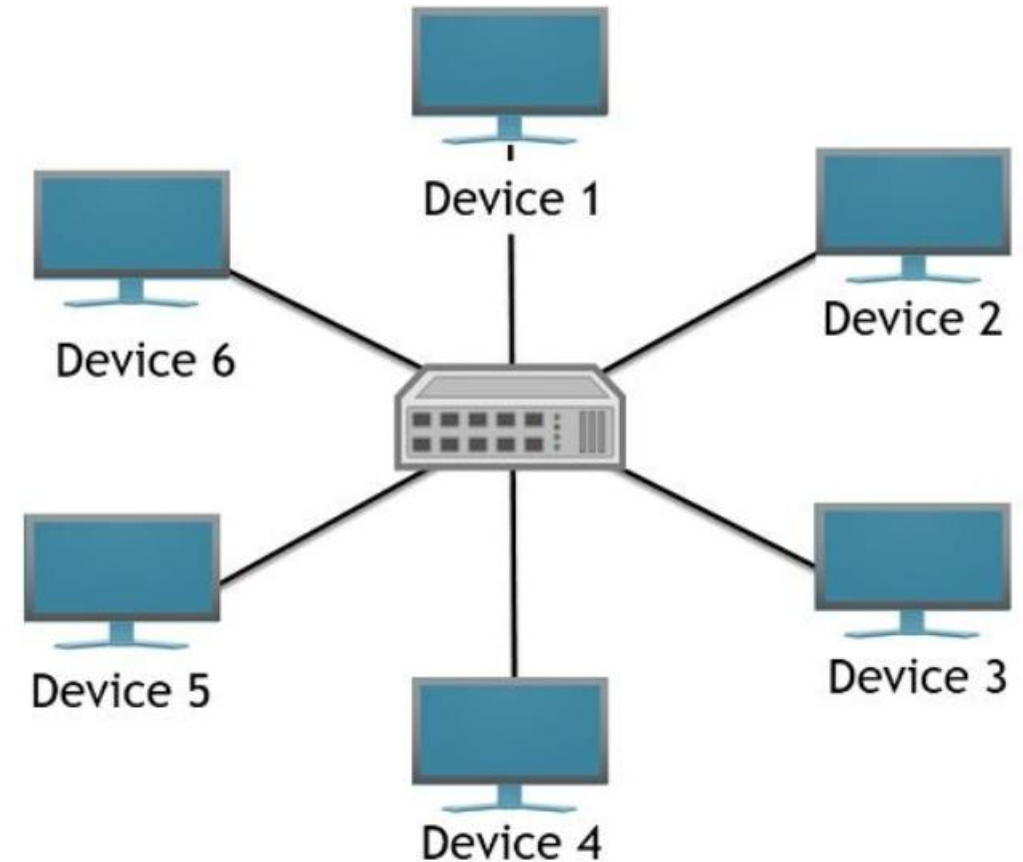
Bus Topology

- All nodes connect to a single shared communication line (the 'bus'), typically via a tap or connector.
- A signal sent by any node travels along the bus and is received by all other nodes.
- **Advantages:**
 - Low cabling cost; simple to install for small networks.
- **Disadvantages:**
 - A single break in the bus can disable the entire network.
 - Performance degrades as more nodes are added (shared medium contention).



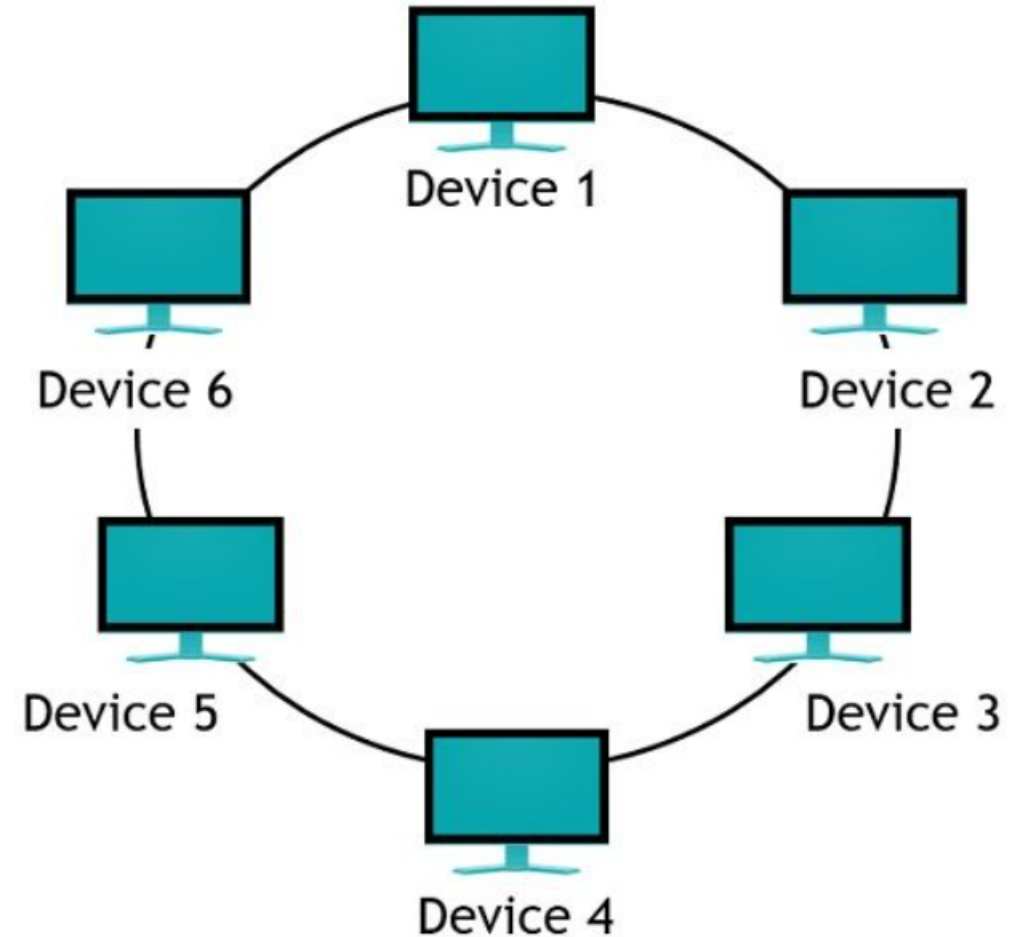
Star Topology

- Every node connects individually to a central device (a hub or switch).
- All traffic passes through the central device, even traffic between two ordinary nodes.
- **Advantages:**
 - Easy to add/remove nodes; a single cable failure only affects one node.
 - Centralized management and troubleshooting.
- **Disadvantages:**
 - The central device is a single point of failure for the whole network.



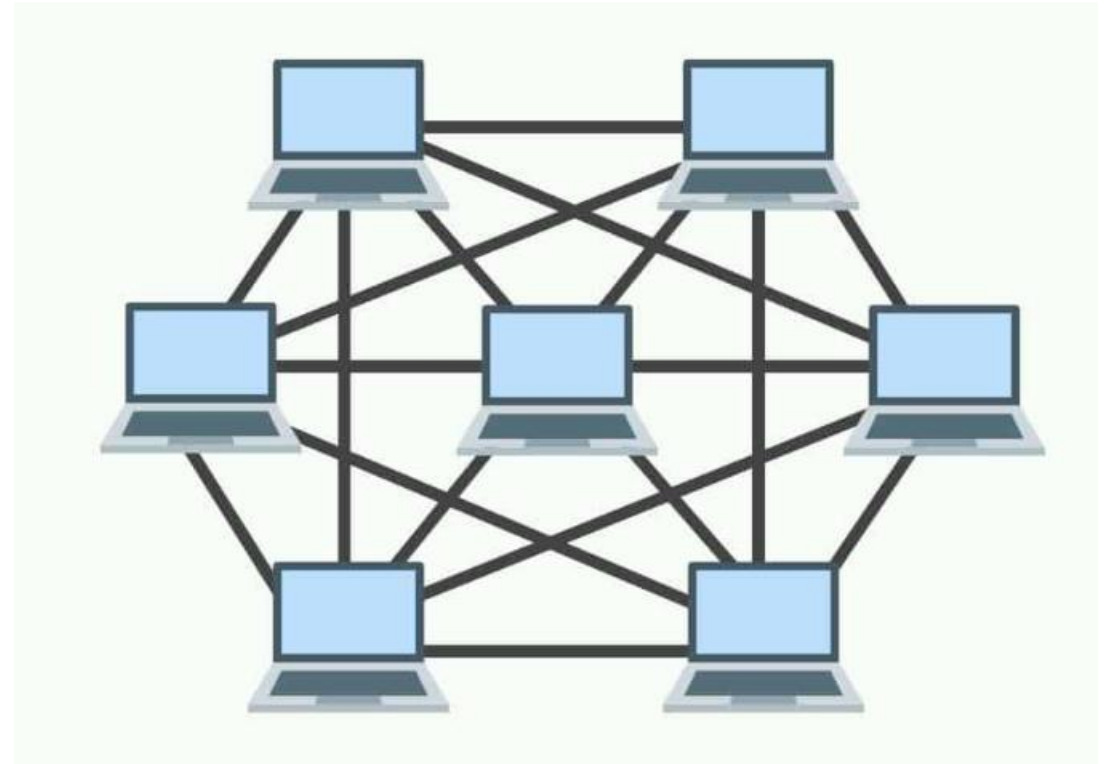
Ring Topology

- Each node connects to exactly two neighbors, forming a closed loop.
- Data travels around the ring, typically in one direction, passing through intermediate nodes to reach its destination.
- **Advantages:**
 - Predictable performance; no collisions if using token passing.
- **Disadvantages:**
 - A single broken link can disrupt the entire ring (unless a dual-ring design is used).
 - Adding or removing a node can disrupt network operation.



Mesh Topology

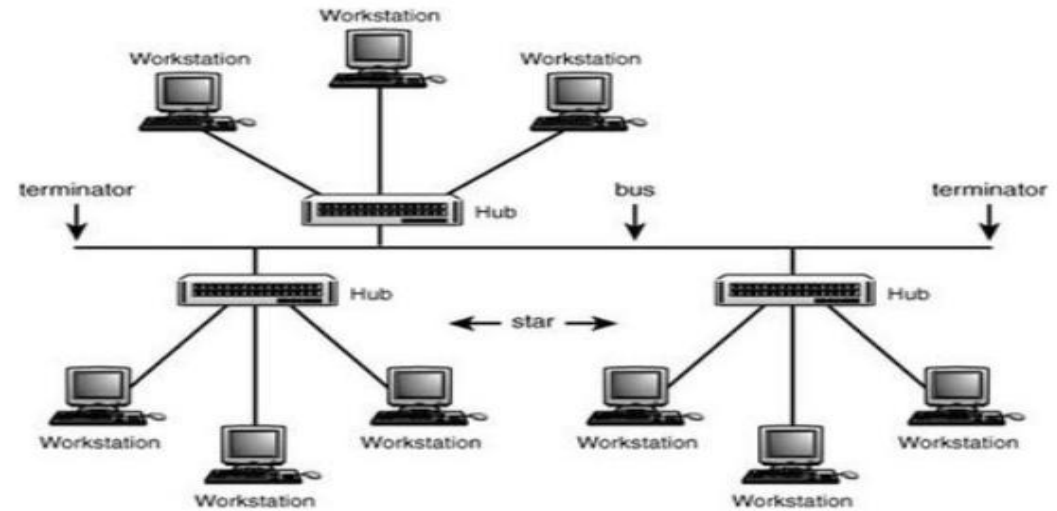
- Full mesh: every node connects directly to every other node.
- Partial mesh: only some nodes have direct links to multiple others.
- **Advantages:**
 - Very high fault tolerance — multiple paths exist between any two nodes.
 - No single point of failure or traffic bottleneck.
- **Disadvantages:**
 - Expensive and complex — a full mesh of n nodes needs $n(n-1)/2$ links.



Tree & Hybrid Topology

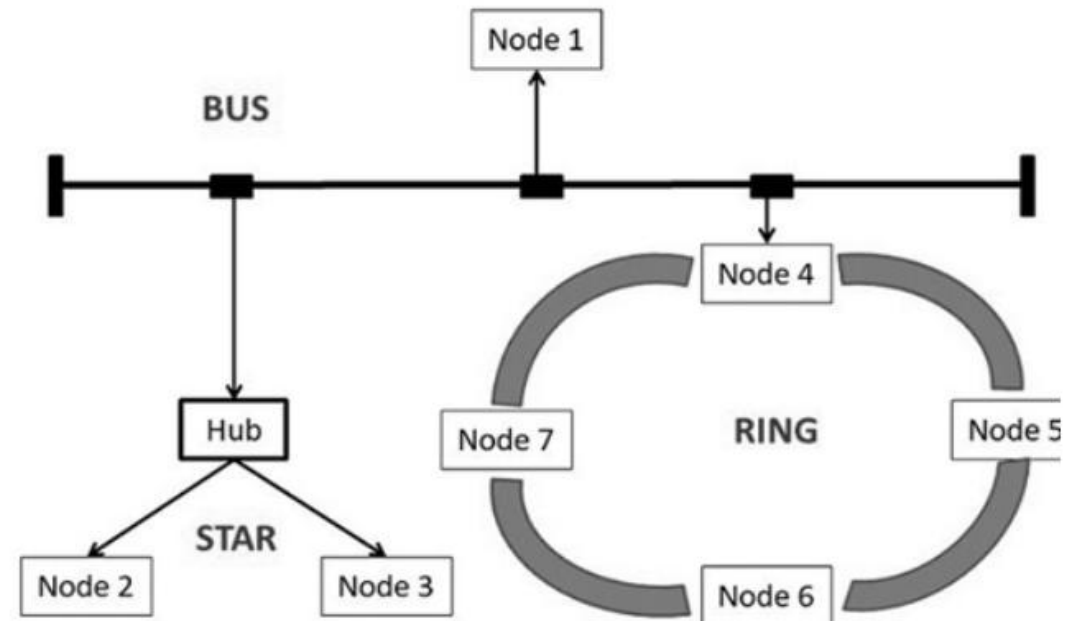
● Tree topology:

- A hierarchical structure — groups of star networks connected to a central backbone, like branches of a tree.
- Scales well for large organizations (e.g. campus networks with per-building stars).



● Hybrid topology:

- Combines two or more basic topologies (e.g. a star-of-rings, or star combined with mesh for critical links).
- Used in practice to balance cost, performance, and reliability for real-world networks.



Topology Comparison

Topology	Cost	Scalability	Fault Tolerance	Typical Use
Bus	Low	Low	Poor (single break disables all)	Small, temporary LANs
Star	Medium	Easy	Good (central point fails whole net)	Most modern LANs/offices
Ring	Medium	Moderate	Moderate (dual-ring improves this)	Legacy token-ring, MANs
Mesh	High	Complex	Excellent (multiple paths)	Backbone/critical infrastructure
Tree	Medium-High	Moderate	Good (isolated branch failures)	Campus / enterprise networks



Design a Campus Network Topology

- **Scenario:** A college campus has **4 buildings** (Admin, Library, Hostel, Labs) that must all be interconnected, and each building has **20–40 devices** to connect internally.
- In groups of **3–4**, sketch a topology for:
 - (a) connecting devices **within each building**, and
 - (b) connecting the **4 buildings** to each other.
- Justify your choice using the comparison table. Consider:
 - Cabling cost
 - Fault tolerance if one building's link fails
 - Ease of adding a **5th building** later
- **Calculate:** If you chose a **full mesh** between the 4 buildings, how many backbone links would you need? (*Hint: $n(n - 1)/2$*)
- Present your design to the class and defend it against **one weakness** a classmate identifies.

TOPIC 3

Encapsulation

How each protocol layer wraps data with its own control information.



What Is Encapsulation?

- When an application hands a message to a protocol (e.g. RRP), that message is just an uninterpreted string of bytes to the protocol.
- The protocol does not care whether the bytes represent an integer array, an email, or an image — it only needs to deliver them.
- But the protocol must still communicate control information to its peer, so it attaches a header to the message.
- **Header:**
 - A small data structure (a few bytes to a few dozen bytes) carrying peer-to-peer control information.
 - Usually placed at the front of the message; if placed at the end, it is called a trailer.
- The original data becomes the body or payload of the new, larger message — this is called encapsulation.

Anatomy of an Encapsulated Message

- A message built by a layer typically has three parts:
 - Header — control info added at the front by this layer's protocol.
 - Payload / body — the data handed down from the layer above (may itself contain a header from that layer).
 - Trailer (optional) — control info some protocols add at the end, e.g. for error checking.
- This repeats at every layer as the message travels down the protocol graph before transmission.



Frame built by HHP, ready for transmission on the wire

Encapsulation Across the Protocol Graph

- Encapsulation repeats at each level of the protocol graph: RRP attaches its header, then HHP encapsulates RRP's entire message by attaching its own header.
- A low-level protocol usually does not interpret the data it is given — it just wraps it. (Some protocols do transform the whole body, e.g. to compress or encrypt it.)
- High-level messages become nested inside low-level messages, like Russian nesting dolls.



Growing outward: each layer adds its own header around what it received from above.

De-encapsulation at the Receiver

- At the destination, the message is processed in exactly the reverse order it was built.
- HHP interprets its header (acts on the control info), then passes the body — without the HHP header — up to RRP.
- RRP interprets its header, then passes the body — without the RRP header — up to the application.
- **Key result:**
 - The message the application receives is identical to the message the sending application handed down — no headers are ever visible to the application layer.
- Intermediate network devices (switches, routers) may inspect lower-layer headers (like the HHP header) without touching the payload above it.



Build and Unwrap an Encapsulated Packet

- ▶ Work in pairs. One partner is the 'Sender stack', the other is the 'Receiver stack'.
- ▶ On paper, write a short message (the 'Application Data'). The Sender wraps it step by step: add an RRP header (write a fake sequence number), then an HHP header (write fake source/destination addresses) around the outside.
- ▶ Pass the fully wrapped 'packet' (the paper) to your partner, the Receiver.
- ▶ The Receiver must de-encapsulate it correctly: read and remove the HHP header first, then the RRP header, and finally recover the original Application Data — in the correct reverse order.
- ▶ Discuss: what would go wrong if the Receiver tried to read the RRP header before removing the HHP header?

TOPIC 4

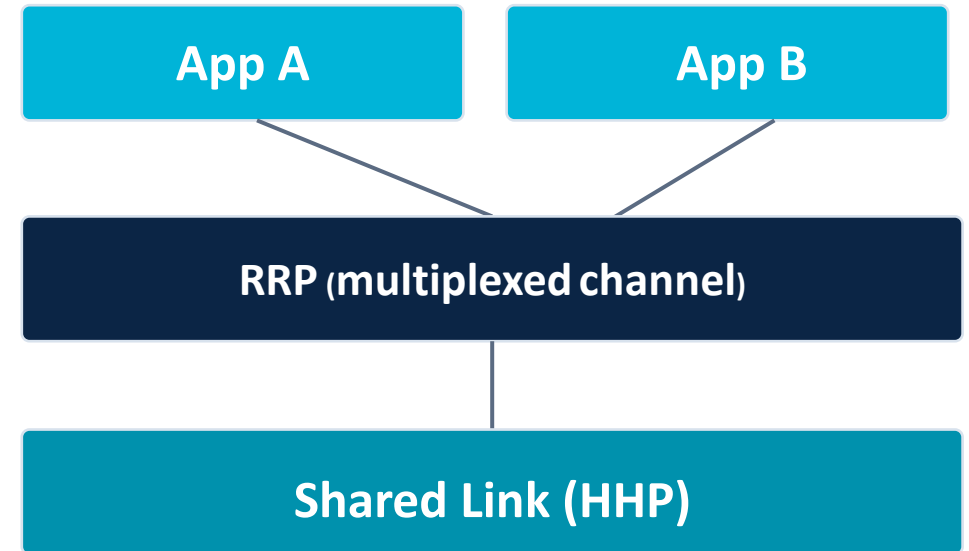
Multiplexing & Demultiplexing

Sharing one channel among many flows, and routing each message back to the right one.



What Is Multiplexing?

- Packet switching's fundamental idea is to multiplex multiple flows of data over a single physical link.
- The same idea applies up and down the protocol graph, not just at switching nodes.
- Example: RRP can implement one logical channel, with messages from two different applications multiplexed over it at the source host.
- At the destination host, those combined messages must then be demultiplexed back to the correct application.



Demultiplexing and the Demux Key

- **Demultiplexing key (demux key):**
 - An identifier included in a protocol's header that records which application (or higher protocol) a message belongs to.
- At the source host, the protocol includes the correct demux key in its header when sending.
- At the destination host, the protocol strips its header, examines the demux key, and delivers the message to the correct application.
- Nearly every protocol supports multiplexing this way — e.g. HHP has its own demux key to decide whether to pass a message up to RRP or to MSP.
- **Important nuance:**
 - There is no universal demux key format — field size (8, 16, or 32 bits) and count (one key vs. a pair of keys) vary between protocols, even within one architecture.

Worked Example: Two Protocols Over HHP

- HHP carries traffic for two higher-level protocols: RRP and MSP.
- Every message HHP delivers includes a demux key identifying whether it belongs to RRP or MSP.
- On arrival, HHP strips its own header, reads the demux key, and hands the message to the correct protocol module (RRP or MSP).
- That protocol then repeats the process one level up, using its own demux key to choose the correct destination application.
- **This is exactly how a modern host uses a single network card to serve many simultaneous applications at once.**

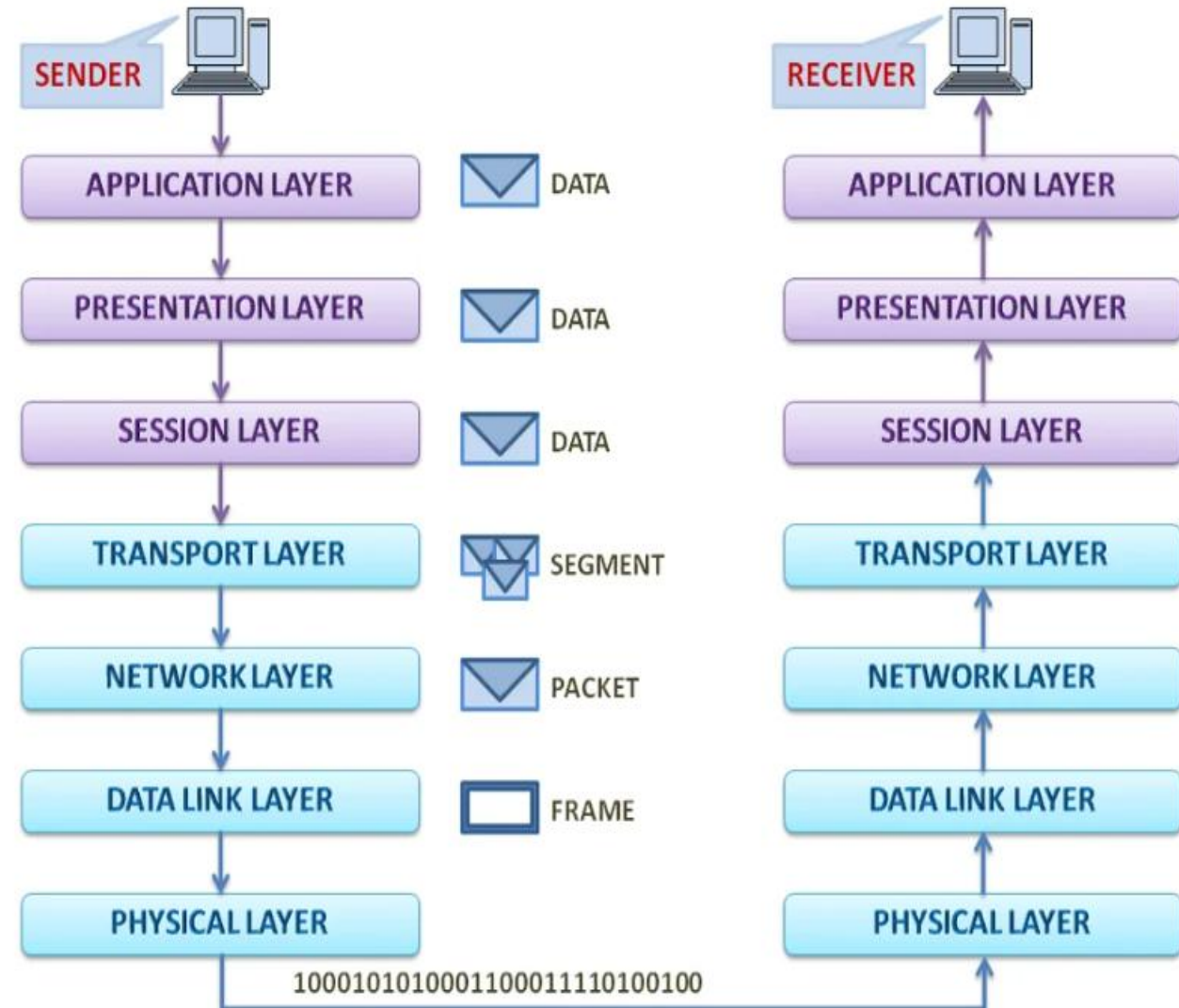
BRAINSTORMING ACTIVITY

Demux Key Detective

- ▶ In small groups, imagine HHP uses an 8-bit demux key field (supporting 256 possible higher-level protocols).
- ▶ Discuss: what problem would arise if two different protocol designers, working independently, each picked the same demux key value for their own protocol?
- ▶ Propose a real-world analogy for a demux key from daily life (e.g., an apartment number on a shared building address, a TV channel number, a radio frequency).
- ▶ Discuss why a protocol might need a PAIR of demux keys (one for each direction) rather than a single shared key — think about how a source and destination might each track many independent connections.
- ▶ Share your best analogy with the class.

The OSI 7-Layer Model — Overview

- ISO's Open Systems Interconnection (OSI) architecture partitions network functionality into 7 layers.
- One or more protocols implement the functionality assigned to each layer.
- It is a reference model for a protocol graph, not a protocol graph itself — no OSI-based network runs today, but its terminology is still widely used.
- The lower 3 layers run on every node (hosts AND switches/routers); the top 4 layers run only on end hosts.



Layers 1-3: Physical, Data Link, Network

- **Layer 1 — Physical:**
 - Handles transmission of raw bits over a communication link.
- **Layer 2 — Data Link:**
 - Collects a stream of bits into a larger aggregate called a frame; implemented by network adaptors and device drivers.
- **Layer 3 — Network:**
 - Handles routing among nodes within a packet-switched network; the unit of data here is a packet.
- **All three of these layers are implemented on every network node — including switches and routers, not just end hosts.**

Layer 4: Transport

- The transport layer implements what we've been calling a process-to-process channel.
- The unit of data exchanged here is commonly called a message, rather than a packet or frame.
- The transport layer — and every layer above it — typically runs only on end hosts, not on intermediate switches or routers.
- This is the layer where reliability, ordering, and flow control between two communicating processes are usually handled.

Layers 5-7: Session, Presentation, Application

- **Layer 5 — Session:**

- Provides a name space tying together potentially different transport streams belonging to one application — e.g. managing an audio stream and a video stream together in a teleconferencing app.

- **Layer 6 — Presentation:**

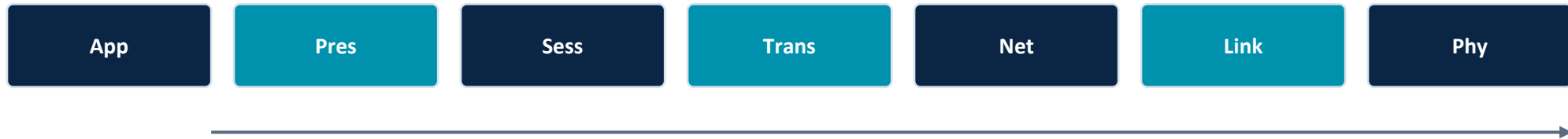
- Concerned with the format of exchanged data — e.g. integer size (16/32/64-bit), byte order, or video stream format.

- **Layer 7 — Application:**

- Where application protocols like HTTP live — the basis of the Web, enabling browsers to request pages from servers.

How Data Flows Through the OSI Stack

- On the sending host, data flows DOWN through the 7 layers — each layer encapsulates the data with its own header (Application → ... → Physical).
- The Physical layer transmits raw bits across the link to the next node.
- On the receiving host, data flows UP through the layers — each layer strips its own header and passes the rest upward (Physical → ... → Application).
- Intermediate devices (like routers) only process up through Layer 3 before forwarding the packet back down and out.



Sender: data travels down, gaining a header at each layer → Physical link → Receiver: data travels up, losing a header at each layer

Why OSI Still Matters Today

- No production network actually runs the full 7-layer OSI stack — the Internet uses the simpler TCP/IP model instead.
- But OSI terminology is universally used by networking professionals: 'Layer 2 switch', 'Layer 3 routing', 'Layer 7 firewall' are everyday industry terms.
- It gives students and engineers a common vocabulary and a clean conceptual reference for discussing where a given protocol or device operates.
- Understanding OSI makes it much easier to understand the more compact, real-world TCP/IP architecture, covered next.

BRAINSTORMING ACTIVITY

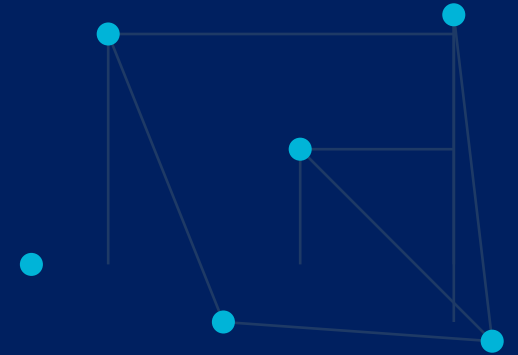
OSI Layer Matching Game

- ▶ The instructor calls out a real-world networking term or scenario one at a time (e.g., 'Wi-Fi radio signal', 'MAC address', 'IP routing table', 'HTTPS encryption format', 'a Zoom call syncing audio and video', 'a web browser requesting a page').
- ▶ In teams, quickly decide which OSI layer (1-7) each term belongs to and write your answer on a whiteboard/paper.
- ▶ Teams reveal answers simultaneously after each term; discuss any disagreements as a class.
- ▶ Bonus round: for one scenario, trace it through ALL 7 layers from Application down to Physical.
- ▶ Wrap-up discussion: which layer was hardest to agree on, and why?

TOPIC 6

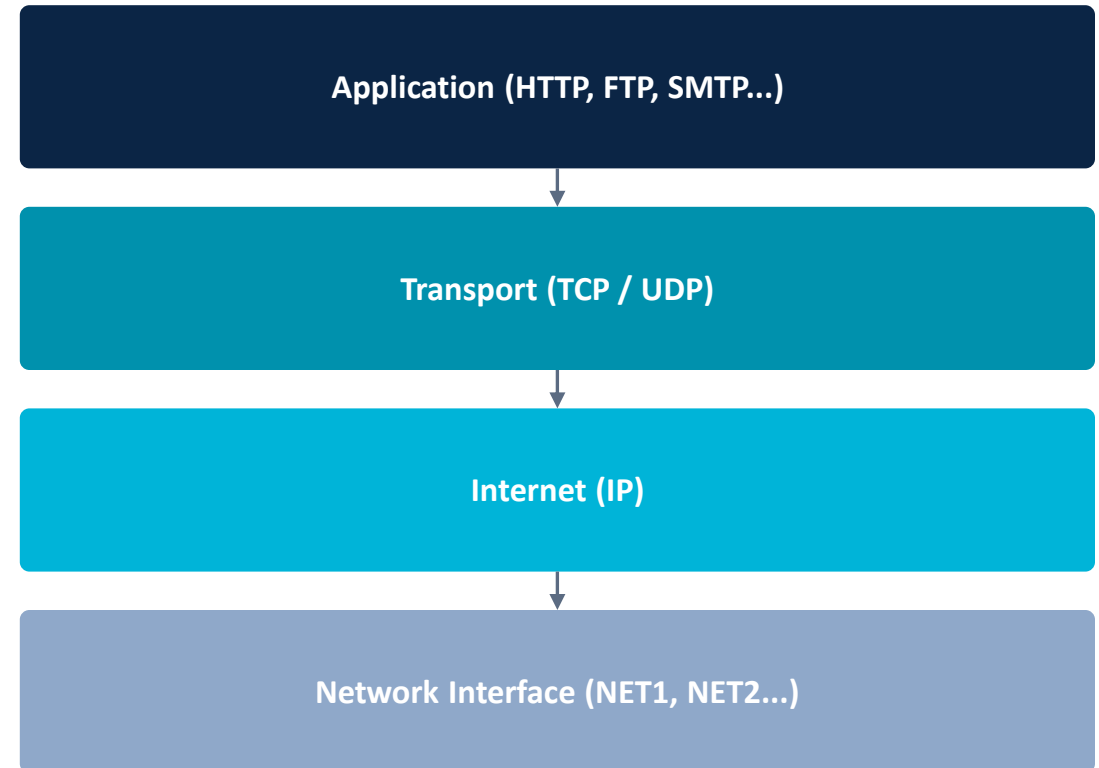
TCP/IP Model

The Internet's own 4-layer architecture, built from real-world experience.



TCP/IP (Internet) Architecture — Overview

- Also called the Internet architecture, named after its two main protocols: TCP and IP.
- Evolved from experience with the ARPANET, an earlier packet-switched network funded by the U.S. Department of Defense's ARPA.
- The Internet and ARPANET predate the OSI model — the experience gained building them directly influenced OSI's design.
- A simpler 4-layer stack is normally used instead of the full 7-layer OSI model.



OSI vs. TCP/IP: Layer Mapping

Layer #	OSI Layer	TCP/IP Layer	Example / Notes
7	Application	Application	HTTP, FTP, SMTP, DNS, Telnet
6	Presentation	(folded into Application)	Data formatting, encryption
5	Session	(folded into Application)	Session/stream management
4	Transport	Transport	TCP (reliable), UDP (unreliable)
3	Network	Internet	IP — the architecture's 'narrow waist'
2	Data Link	Network Interface	Ethernet, Wi-Fi (802.11)
1	Physical	Network Interface	Cabling, radio signals

TCP vs. UDP

Aspect	TCP	UDP
Channel type	Reliable byte-stream	Unreliable datagram delivery
Delivery guarantee	Guaranteed, in-order delivery	Best-effort, no guarantee
Overhead	Higher (acknowledgements, retransmission)	Lower (minimal header)
Typical use	Web (HTTP), email, file transfer	Streaming, DNS queries, gaming, VoIP

Both TCP and UDP are called end-to-end protocols (or transport protocols) — they provide alternative logical channels directly to application programs, both running on top of IP.

Application Layer Protocols

- A range of application protocols run above the transport layer, enabling interoperation of popular applications:
 - HTTP — Hypertext Transfer Protocol, the basis of the World Wide Web.
 - FTP — File Transfer Protocol.
 - Telnet — remote login.
 - SMTP — Simple Mail Transfer Protocol, for email.
- Different browsers (Chrome, Firefox, Safari) all interoperate with any web server because they conform to the same HTTP specification.
- **Note the terminology overlap:**
 - The same term (e.g. 'FTP') often names both the application program AND the protocol it implements.

Trace a Protocol Stack

- ▶ Scenario: A student opens a web browser and requests a page from a college website using HTTP.
- ▶ In groups, write out the full protocol stack this request travels through, from Application down to Network Interface, labeling each TCP/IP layer.
- ▶ Decide: should this request use TCP or UDP at the transport layer? Justify your choice using the TCP vs UDP comparison table.
- ▶ Now repeat the exercise for a live video call (e.g. a VoIP or video-conferencing app) — does your transport-layer choice change? Why?
- ▶ Map both scenarios onto the OSI 7-layer model as well, and note which OSI layers TCP/IP compresses together.

TOPIC 7

Basics of the Physical Layer

How raw bits actually travel across cables, fibers, and the air.



Role of the Physical Layer

- The physical layer is Layer 1 — it handles the transmission of raw bits (0s and 1s) over a communication link.
- It defines the electrical, mechanical, and timing details of turning bits into signals: voltage levels, light pulses, or radio waves.
- It does not understand frames, addresses, or messages — it simply moves a stream of bits from one point to another.
- Everything above it (data link and up) depends on the physical layer successfully delivering bits, even if imperfectly — higher layers handle error detection and correction.

Transmission Media: Guided (Wired)

- **Twisted Pair Cable:**

- Copper wires twisted together to reduce interference; inexpensive; used in Ethernet LANs (Cat5e/Cat6).

- **Coaxial Cable:**

- A central conductor shielded by a metal mesh; more resistant to interference than twisted pair; used in cable TV/broadband.

- **Fiber Optic Cable:**

- Transmits data as pulses of light through glass/plastic fiber; extremely high bandwidth, very low signal loss, immune to electromagnetic interference; used for backbones and long-haul links.

Transmission Media: Unguided (Wireless)

- **Radio Waves:**

- Used by Wi-Fi (802.11 standards) and cellular networks; can penetrate walls; range varies with frequency and power.

- **Microwaves:**

- Line-of-sight transmission; used for point-to-point links and satellite communication.

- **Infrared:**

- Short-range line-of-sight; used in remote controls and some short-range device links.

- Wireless media are shared, so multiple devices must coordinate access to avoid interference — a key challenge unique to unguided transmission.

Key Concepts: Bandwidth, Signals, Bit Rate

Bandwidth

The range of frequencies a medium can carry, or loosely, the maximum data rate a link can support — measured in Hz (frequency) or bits per second (data capacity).

Bit Rate

The number of bits transmitted per second (bps) — determines how fast data actually moves across the link.

Analog vs Digital Signal

Analog signals vary continuously (e.g. traditional telephone voice); digital signals take discrete values (0s and 1s) — most modern networks are digital.

Attenuation & Noise

Attenuation is signal weakening over distance; noise is unwanted interference — both physical layer engineers must design around.

Media Selection Challenge

- ▶ In groups, you are network consultants. For each scenario below, choose the best transmission medium and justify it in one sentence:
- ▶ (a) Connecting two office buildings 2 km apart, needing very high bandwidth and immunity to lightning/electrical interference.
- ▶ (b) Providing internet access to student laptops moving freely around a hostel common room.
- ▶ (c) Connecting a desktop PC to a switch 5 meters away in a low-interference office.
- ▶ (d) Linking two ground stations to a satellite in orbit.
- ▶ Discuss: which choice would change if cost was the ONLY factor instead of performance?

TOPIC 8

Internet Architecture

The hourglass philosophy that let the Internet scale and evolve.



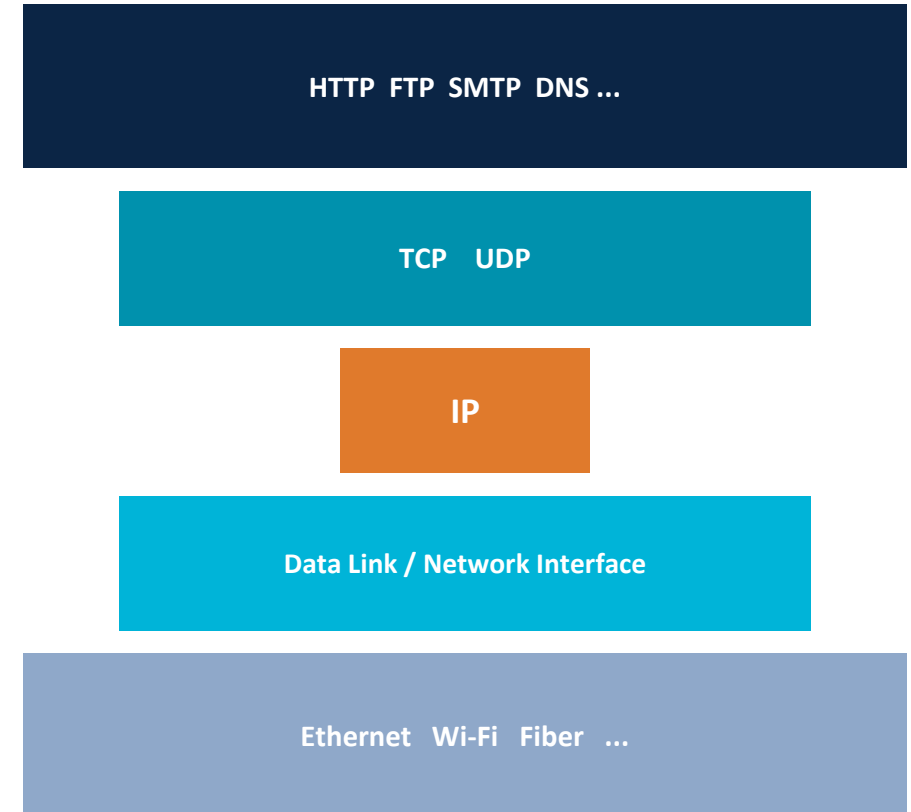
Origins: From ARPANET to the Internet

- The Internet architecture evolved from experience with the ARPANET, an earlier packet-switched network.
- Both the ARPANET and the Internet were funded by ARPA (Advanced Research Projects Agency), part of the U.S. Department of Defense.
- The Internet and ARPANET existed before the OSI architecture — lessons learned building them heavily influenced the OSI reference model, not the other way around.
- The Internet architecture is also called the TCP/IP architecture, named after its two most central protocols.

The Hourglass Model



- The Internet protocol graph has an hourglass shape: wide at the top, narrow in the middle, wide at the bottom.
- IP sits at the narrow waist — a single, common method for exchanging packets across many different networks.
- Above IP: arbitrarily many transport protocols (TCP, UDP, ...), each offering a different channel abstraction to applications.
- Below IP: arbitrarily many network technologies (Ethernet, Wi-Fi, fiber, ...) can be used interchangeably.
- **This separates 'delivering messages host-to-host' from 'providing a useful process-to-process service' — the central philosophy of the architecture.**



Narrow waist = one common protocol (IP) everything above and below can rely on.

Three Key Features of Internet Architecture

- **1. No strict layering enforced:**
 - Applications are free to bypass defined transport layers and use IP (or even an underlying network) directly; programmers can define entirely new channel abstractions.
- **2. The hourglass philosophy:**
 - IP is the focal point — a common way to exchange packets across many networks, cleanly separating host-to-host delivery from process-to-process service.
- **3. Specification + working implementation required:**
 - For the IETF to adopt a new protocol, there must be both a written specification AND at least one (preferably two) working implementations.

IETF & 'Rough Consensus and Running Code'

- The IETF (Internet Engineering Task Force) is the primary standardization body for the Internet architecture and many of its core protocols (TCP, UDP, IP, DNS, BGP).
- The Internet architecture also embraces protocols from other bodies: IEEE (802.3 Ethernet, 802.11 Wi-Fi), W3C (HTTP/HTML), 3GPP (4G/5G), ITU-T (H.264 video).
- IANA (Internet Assigned Numbers Authority), a department of ICANN, hands out the unique identifiers protocols need to interoperate.

*“We reject kings, presidents, and voting.
We believe in rough consensus and running code.”*
— David Clark, widely quoted at IETF meetings

BRAINSTORMING ACTIVITY

Redesign the Hourglass

- ▶ In groups, discuss: why is it important that the 'waist' of the hourglass (IP) stays narrow — i.e., why not add lots of features directly into IP itself?
- ▶ Brainstorm: what would happen to Internet innovation if a new physical network technology required rewriting every application? Connect this to the hourglass's separation of concerns.
- ▶ Discuss one modern technology that sits ABOVE the transport layer (e.g. a new streaming protocol) and one that sits BELOW IP (e.g. a new wireless standard) — explain why neither required changes to IP itself.
- ▶ Debate: is a narrow, stable waist still the right design for the Internet today, given the huge diversity of modern devices (IoT, mobile, satellites)? Give one argument for and one against.

Self-Assessment: Practice Questions

- 1. Explain the difference between a protocol's service interface and its peer interface, using an example.
- 2. Why does the Internet architecture avoid enforcing strict layering, unlike the OSI model?
- 3. Draw the encapsulated message that results when application data passes through RRP and then HHP.
- 4. Compare a star topology and a mesh topology in terms of cost and fault tolerance.
- 5. What is a demux key, and why can't all protocols agree on one universal format for it?
- 6. Map each OSI layer to its corresponding TCP/IP layer, noting which OSI layers are combined.
- 7. Explain why IP is described as the 'narrow waist' of the Internet's hourglass architecture.
- 8. List two guided and two unguided transmission media, with one use case each.

Application Programming Interface (Sockets)

API (Application Programming Interface) acts as a bridge between an application and network services.

It enables applications to send requests and receive responses over a network using standard interfaces.

Software Makes the Internet Powerful

- **Why is the Internet so successful?**
 - The Internet became successful not only because of cables and routers but because of software.
 - Software allows developers to create new applications without changing the Internet infrastructure.

Examples

- WhatsApp
- YouTube
- Gmail
- Zoom
- Google Maps
- Instagram

All these applications work because they use the same Internet.

What is a Network Application?

- A Network Application is software that communicates with another software over a network.

Examples

Application	Communicates With
WhatsApp	WhatsApp Server
Chrome	Web Server
Gmail	Mail Server
Zoom	Zoom Server
PUBG	Game Server

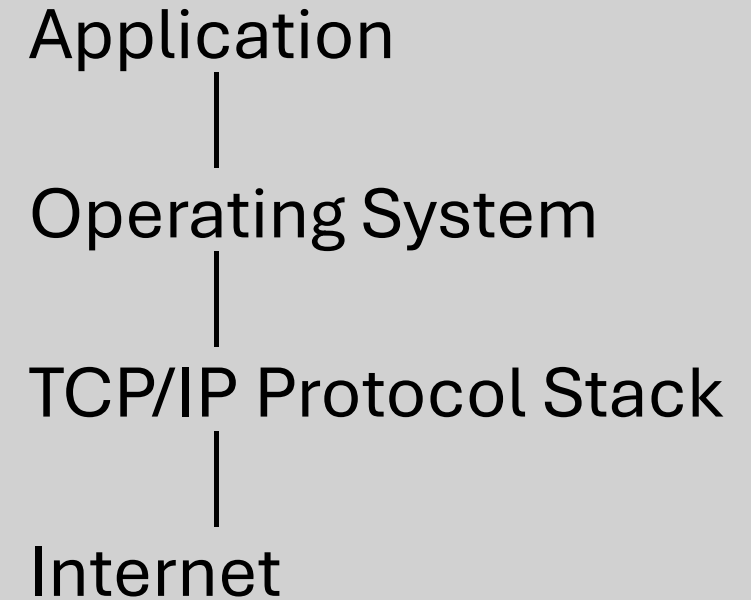
Real-life Example

When you send "Hello" through WhatsApp
your phone application
↓
uses the Internet
↓
reaches the WhatsApp server
↓
server sends it to your friend's phone.

How Applications Communicate?

- Applications do NOT
 - touch Ethernet cables
 - control routers
 - manage switches

Instead, they ask the Operating System for networking services.



What is an API ?

- API = Application Programming Interface
 - An API is a set of functions provided by the Operating System.
 - Applications use these functions to communicate over the network.

Example

- Instead of building networking from scratch, the application simply asks the OS "Please send this message."
- The Operating System handles everything.

The Network API Concept

- Every OS provides an interface to its networking subsystem → Network API
- Most protocols are implemented in software within the OS

What is Socket API?

- **A Socket API is a programming interface used by applications to communicate over a network.**

- ❖ It acts as the connection point between Application and Network.

- **Why are Sockets Important?**

- ❖ Without sockets, every programmer would need to build networking from scratch.
 - ❖ Sockets make networking easy.
 - ❖ They provide ready-made functions.

- **Benefits**

- ❖ Easy programming
 - ❖ Faster application development
 - ❖ Same interface on Windows, Linux and Mac
 - ❖ Used in Java, Python and C

- **Socket API:** Originated in Berkeley Unix (BSD)
- Now supported by virtually all major OSs
- Foundation for language-specific libraries (Java, Python sockets, etc.)

Significance of the Socket API

- **Defines the boundary between:**
 - Applications running on top of the Internet
 - The internal implementation of the Internet
- A stable, well-defined interface → enabled Internet applications to become a multi-billion-dollar industry
- Evolution: simple client/server apps (email, FTP, remote login) → today's cloud applications on smartphones

Concept	Description
Protocol	Defines the <i>services</i> offered (e.g., TCP's reliable byte stream)
API	Defines the <i>syntax</i> to invoke those services on a system

The Socket - Core Abstraction

- ❖ **Definition:** The point where a local application process attaches to the network
- Socket interface provides operations to:
 - **Create** a socket
 - **Attach** it to the network
 - **Send/receive** messages
 - **Close** the socket
- Focus: Sockets used with **TCP** (connection-oriented)

Creating a Socket

```
int socket(int domain, int type, int protocol);
```

domain → protocol family

PF_INET – Internet family

PF_UNIX – Unix pipe facility

PF_PACKET – direct network interface access (bypasses TCP/IP)

type → communication semantics

SOCK_STREAM – byte stream (TCP)

SOCK_DGRAM – message-oriented (UDP)

protocol → specific protocol (often UNSPEC; PF_INET + SOCK_STREAM implies TCP)

Return value: socket handle (identifier for future operations)

Server Side - Passive Open

Server prepares to *accept* connections (doesn't establish one itself)

Three operations:

```
int bind(int socket, struct sockaddr *address, int addr_len);  
int listen(int socket, int backlog);  
int accept(int socket, struct sockaddr *address, int *addr_len);
```

bind → attaches socket to local address (IP + **port number**)

Ports act as demultiplexing keys

Well-known ports (e.g., HTTP = port 80)

listen → sets max number of pending connections

accept → blocking call; waits for a client to connect

Returns a **new socket** for the established connection

Original socket remains open for future accept() calls

Client Side - Active Open

```
int connect(int socket, struct sockaddr *address, int addr_len);
```

- Client specifies the address of the remote server it wants to connect to
- Blocks until TCP successfully establishes the connection
- Client generally lets the OS choose its own local port (doesn't care which one)
- Server, in contrast, listens on a well-known fixed port

Sending and Receiving Data

```
int send(int socket, char *message, int msg_len, int flags);  
int recv(int socket, char *buffer, int buf_len, int flags);
```

- **send()** → transmits a message over the connected socket
- **recv()** → receives a message into a buffer
- Both accept **flags** to control operation details
- Used after connection is successfully established (client & server)

Socket API - Summary

Operation	Used By	Purpose
socket()	Both	Create a socket
bind()	Server	Attach socket to local address/port
listen()	Server	Set pending connection queue size
accept()	Server	Accept incoming connection (blocking)
connect()	Client	Initiate connection to server
send()	Both	Send data
recv()	Both	Receive data
close()	Both	Terminate the connection

Summary

- The Socket API is the demarcation point between applications and the underlying network implementation
- Sockets = local endpoint of a network connection
- TCP socket programming involves a clear sequence: socket → bind/listen/accept (server) and socket → connect (client)
- Same API pattern underlies most modern client-server and networked applications, including cloud services

Network Performance Metrics

Throughput • Bandwidth • Delay • Latency • High-Speed Networks

Why Network Performance Matters

"First get it right, then make it fast"

is a classic programming adage but in networking, performance must often be

designed in from the start.

- The effectiveness of distributed computations depends directly on how efficiently the network delivers data.
- Network performance is judged using two fundamental measures: bandwidth (throughput) and latency (delay).
- Understanding these metrics lets engineers predict, diagnose, and improve real-world application behaviour over a network.

TWO PILLARS OF PERFORMANCE

Bandwidth

Volume of data delivered per unit time (bits/second).

Latency

Time taken for a message to travel end-to-end.

Together, they define the complete performance profile of any link or channel.

Bandwidth: The Data Highway

Definition

The number of bits that can be transmitted over a network in a given period of time.

*e.g. a 10 Mbps network delivers 10 million bits every second
each bit takes $0.1\ \mu\text{s}$ to transmit.*

Two Senses of the Word

Signal bandwidth (Hz):

width of a frequency band e.g. voice-grade phone line: 3300 Hz – 300 Hz = 3000 Hz.

Data-link bandwidth (bits/sec):

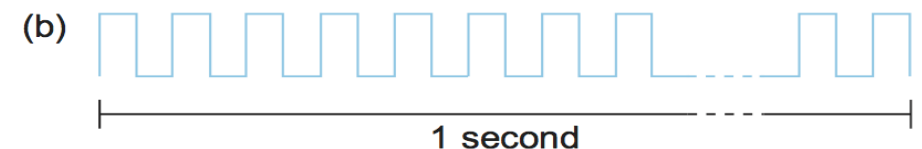
the commonly used networking sense e.g. "a 10 Mbps Ethernet link."

VISUALIZE

Think of 1 second as a length of ruler.
Bandwidth = how many bit-pulses fit in that length.

1 Mbps $\rightarrow$ each bit is $1\ \mu\text{s}$ wide

2 Mbps $\rightarrow$ each bit is $0.5\ \mu\text{s}$ wide



Bandwidth vs Throughput vs Requirement

Term	What it Means	Example
Bandwidth (Data Rate)	Maximum data rate available on a link, a theoretical capacity.	"The link bandwidth is 10 Mbps."
Throughput	Actual measured performance achieved in practice, often lower than bandwidth due to inefficiencies.	A 10 Mbps link achieving only 2 Mbps throughput.
Bandwidth Requirement	Bits/second an application needs to perform acceptably may be fixed, variable, or "best-effort."	A video app needing a steady 5 Mbps.

Bandwidth is the capacity of the pipe; throughput is what actually flows through it. The gap between the two is caused by protocol overhead, congestion, and implementation inefficiencies.

What Influences Bandwidth?

1

Physical Link Bandwidth

Constantly improving with better transmitting/receiving technology narrower bit-pulses fit more bits per second.

2

Logical Channel Bandwidth

Process-to-process bandwidth is also shaped by how many times software must handle or transform each bit.

3

Bit Width

Each bit occupies a fixed 'width' in time e.g. 1 μ s at 1 Mbps, 0.5 μ s at 2 Mbps. Higher bandwidth = narrower bits.

Example

On a 10-Mbps link: Time to transmit 1 bit = $1 / (10 \times 10^6) = 0.1 \mu$ s.

On a 100-Mbps link: Time to transmit 1 bit = $1 / (100 \times 10^6) = 0.01 \mu$ s — bits are 10 $\times$ narrower.

Bandwidth vs Throughput — Real Life

- 1 Your ISP advertises "100 Mbps" broadband, but a speed test shows only 62 Mbps. List THREE possible real-world causes for this gap.
- 2 A file-sharing app needs "whatever bandwidth it can get," while a video call needs a fixed 3 Mbps. Classify 3 more everyday apps by their bandwidth requirement type.
- 3 Quick Poll: Would doubling your Wi-Fi router's advertised bandwidth double your real download speed? Justify your answer using today's concepts.

Latency: The Time Cost of a Journey

Definition

Latency (delay) is how long it takes a message to travel from one end of a network to the other.

Example: A transcontinental network may have a one-way latency of 24 ms.

Round-Trip Time (RTT)

The time to send a message from one end to the other AND back , often more important than one-way latency in practice (e.g. request–response protocols, TCP handshakes).

REMEMBER

Latency $\neq$ Bandwidth

Latency is measured strictly in units of TIME (ms, μ s) , never in bits/second.

One-way latency

Source $\rightarrow$ Destination, single direction

Round-Trip Time

Source $\rightarrow$ Destination $\rightarrow$ back to Source

The Three Components of Latency

1

Propagation Delay

Time for a signal to physically travel through the medium — limited by the speed of light in that medium.

2

Transmit Delay

Time needed to push all bits of a packet onto the link — depends on packet size and bandwidth.

3

Queuing Delay

Time a packet waits inside switches/routers before being forwarded on an outbound link.

$$\text{Latency} = \text{Propagation} + \text{Transmit} + \text{Queue}$$

Latency Formulas

Propagation Delay

$$\text{Propagation} = \text{Distance} / \text{SpeedOfLight}$$

Distance = length of the wire; SpeedOfLight = effective speed in that medium.

Transmit Delay

$$\text{Transmit} = \text{Size} / \text{Bandwidth}$$

Size = packet size (bits); Bandwidth = link's transmission rate.

$$\text{Total Latency} = \text{Propagation} + \text{Transmit} + \text{Queue}$$

Special case: for a single link and a 1-bit message, Transmit and Queue vanish — latency = propagation delay only.

"Latency" and "delay" are used generically in networking; "propagation delay" refers specifically to signal travel time.

Speed of Light Varies by Medium

Medium	Effective Speed of Light	Relative to Vacuum
Vacuum / Free Space	$3.0 \times 10^8 \text{ m/s}$	100% (baseline)
Copper Cable	$2.3 \times 10^8 \text{ m/s}$	~77%
Optical Fiber	$2.0 \times 10^8 \text{ m/s}$	~67%

Don't plug in the vacuum speed of light ($3 \times 10^8 \text{ m/s}$) for every problem. Always check the transmission medium stated in the question fiber and copper are noticeably slower than free space, and this changes your propagation-delay answer.

Formula reminder: Propagation Delay = Distance ÷ Speed-of-Light (in that medium)

Worked Example: Propagation Delay

Problem: An optical-fiber link runs 1,000 km between two cities. Find the propagation delay.

Step 1

Identify values: Distance = 1000 km = 1×10^6 m; Speed in fiber = 2.0×10^8 m/s

Step 2

Apply formula: Propagation = Distance / SpeedOfLight

Step 3

Compute: Propagation = $(1 \times 10^6) / (2.0 \times 10^8) = 5 \times 10^{-3}$ s = 5 ms

Answer: Propagation Delay = 5 milliseconds (one-way)

When Does Latency Dominate?

Case: Small Messages (Latency-Bound)

A client sends a 1-byte message and receives a 1-byte reply , think of a keystroke echoed back, or a single ping.

Across the Room

RTT = 1 ms

Transmit time @ 1 Mbps for 1 byte = 8 μ s

Transmit time @ 100 Mbps = 0.08 μ s

→ *Response time $\approx$ 1 ms either way*

Transcontinental

RTT = 100 ms

Same negligible transmit times, but the round-trip time itself dominates completely.

→ *Link speed (1 Mbps vs 100 Mbps) barely matters here!*

When Does Bandwidth Dominate?

Case: Large Transfers (Bandwidth-Bound) fetching a 25 MB digital-library image over a 10 Mbps channel.

Transmit Time = Size / Bandwidth = $(25 \times 10^6 \times 8 \text{ bits}) / (10 \times 10^6 \text{ bps}) = 20 \text{ seconds}$

1-ms RTT channel

Total ≈ 20.001 seconds

RTT contributes a tiny 0.001 s

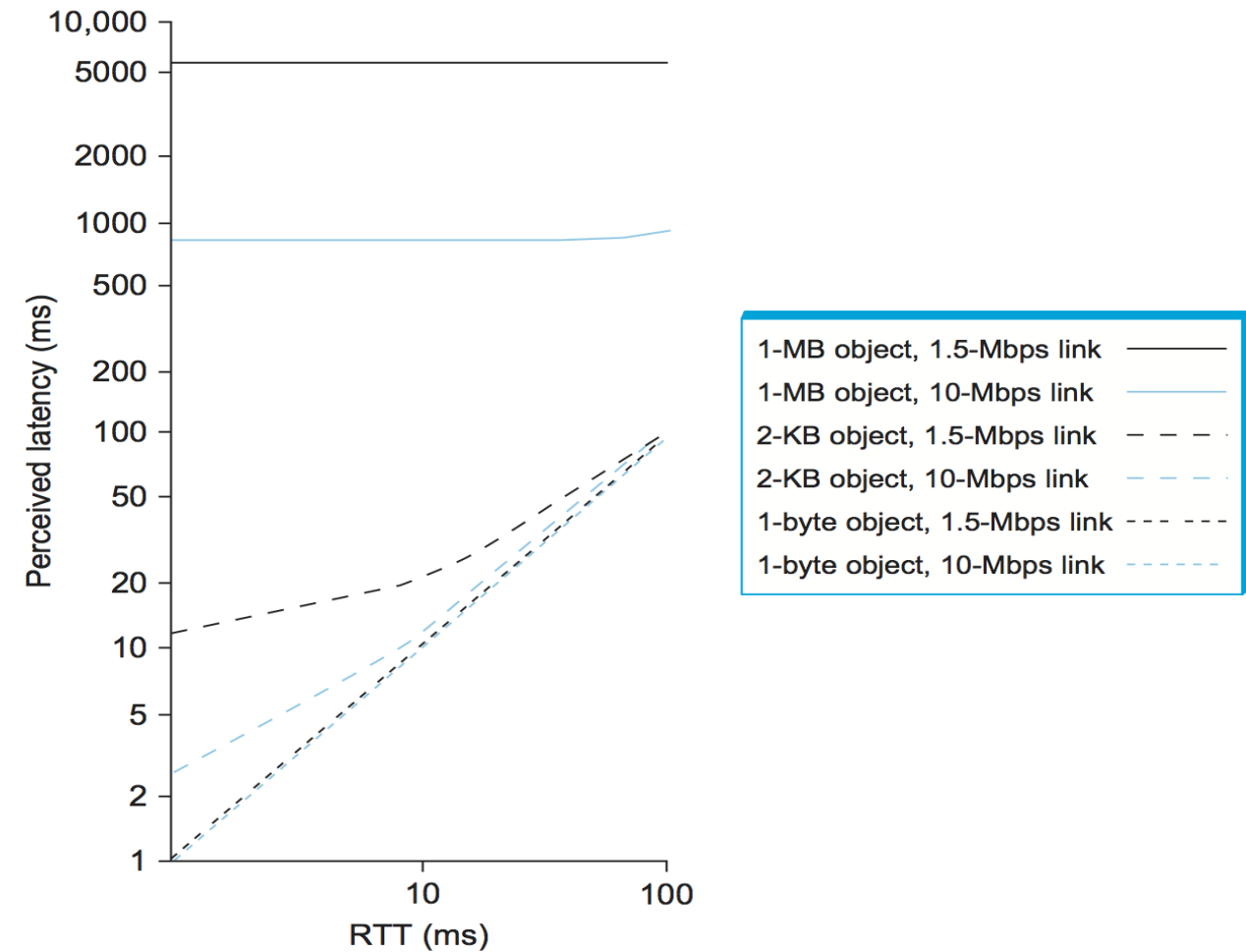
100-ms RTT channel

Total ≈ 20.1 seconds

Negligible difference from the 1-ms case!

Visualizing Latency vs Bandwidth Dominance

- 1-byte object: response time $\approx$ RTT, 1.5 Mbps vs 10 Mbps link makes no visible difference.
- 2-KB object (an email): link speed matters a lot at low RTT, negligibly at high RTT.
- 1-MB object (an image): RTT barely matters, link (bandwidth) speed dominates across the board.



Perceived latency (response time) versus round-trip time for various object sizes and link speeds.

Latency-Bound or Bandwidth-Bound?

- 1 An online multiplayer game sends tiny 20-byte position updates 30 times/second. Is this game latency-bound or bandwidth-bound? Explain.
- 2 A cloud backup service uploads a 4K video (2 GB) overnight. Would improving your RTT from 80 ms to 20 ms noticeably speed this up? Justify with reasoning from today's examples.
- 3 Brainstorm: Name ONE real application from your daily life for each category — (a) latency-bound, (b) bandwidth-bound, (c) roughly balanced.

The Delay $\times$ Bandwidth Product

Definition

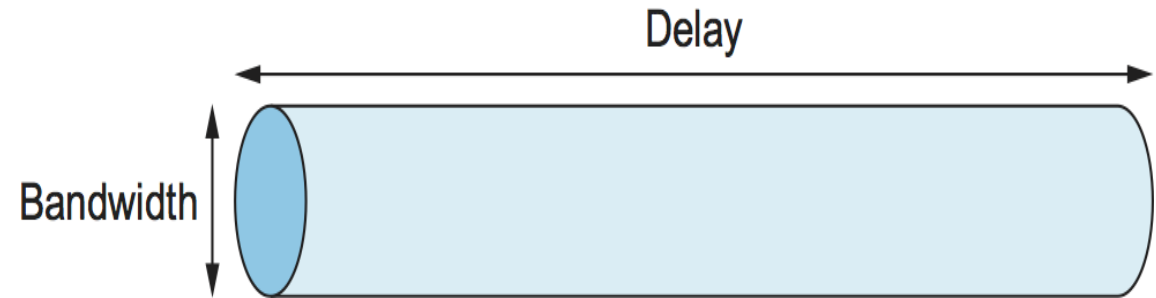
The product of latency and bandwidth, the maximum number of bits that can be "in flight" (in transit) through a channel at any instant.

The Pipe Analogy

Latency = length of the pipe

Bandwidth = diameter of the pipe

Delay $\times$ Bandwidth = volume of the pipe (bits it can hold)



Computing the Delay × Bandwidth Product

$$\text{Delay} \times \text{Bandwidth Product} = \text{Latency (RTT or one-way)} \times \text{Bandwidth}$$

Example

A transcontinental channel has one-way latency = 50 ms and bandwidth = 45 Mbps.

$$\begin{aligned}\text{Volume} &= 50 \times 10^{-3} \text{ s} \times 45 \times 10^6 \text{ bits/s} \\ &= 50 \times 45 \times 10^{-3+6} \\ &= 2250 \times 10^3 \text{ (or using scientific notation)} \quad 2.25 \times 10^6 \text{ bits} \\ &= 2.25 \times 10^6 / 8 \text{ (To convert to byte)} \\ &= 281250 \text{ Bytes} \approx 280 \text{ KB}\end{aligned}$$

If we instead use RTT ($2 \times 50 \text{ ms} = 100 \text{ ms}$): $\text{RTT} \times \text{Bandwidth} = 100 \times 10^{-3} \times 45 \times 10^6 = 5.5 \times 10^6 \text{ bits} \approx 671 \text{ KB}$, the amount that could arrive after a "stop" signal is sent, before the sender reacts.

Table 1: RTT × Bandwidth for Real Networks

Link Type	Bandwidth	One-Way Distance	RTT	RTT × Bandwidth
Wireless LAN	54 Mbps	50 m	0.33 μs	18 bits
Satellite	1 Gbps	35,000 km	230 ms	230 Mb
Cross-country Fiber	10 Gbps	4,000 km	40 ms	400 Mb

Interpreting the Table

• **Wireless LAN:**

Tiny RTT × Bandwidth , almost nothing is ever "in flight."

• **Satellite:**

Huge product hundreds of megabits can be in transit simultaneously due to the very long RTT.

• **Cross-country Fiber:**

High bandwidth combined with moderate delay also yields a very large product.

Compute the Delay $\times$ Bandwidth Product

- 1 A satellite link has a one-way propagation delay of 270 ms and bandwidth of 2 Mbps. Compute the RTT $\times$ Bandwidth product in bits and in KB.
- 2 If the sender in Q1 wants to keep the pipe completely full (never idle waiting for an ACK), how many bits should it send before pausing?
- 3 Brainstorm: Why might a satellite link need a much larger 'window' of unacknowledged data in flight than a LAN link? Connect your answer to the pipe analogy.

High-Speed Networks: What Changes, What Doesn't

WHAT DOES NOT CHANGE

The Speed of Light

A transcontinental RTT is the same ~100 ms whether the link is 1 Mbps or 1 Gbps propagation delay is fixed by distance and medium, not by bandwidth.

WHAT DOES CHANGE

Bandwidth Keeps Rising

As bandwidth rises without bound, our thinking about network design must shift:

Going from 100 RTTs to 101 RTTs (1% more) feels trivial. But going from 1 RTT to 2 RTTs (100% more) is a huge relative jump so latency, not throughput, starts to dominate our thinking.

Example: A 1 MB File on Two Networks

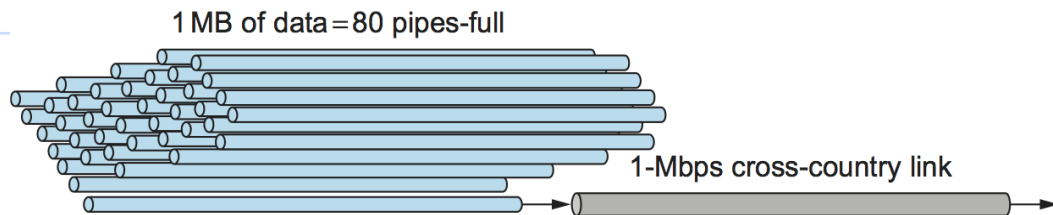
Both networks below share the same 100 ms RTT.

1-Mbps Network

80 round-trip times

needed to transmit the file — each RTT carries only 1.25% of it.

The file behaves like a continuous STREAM of data.

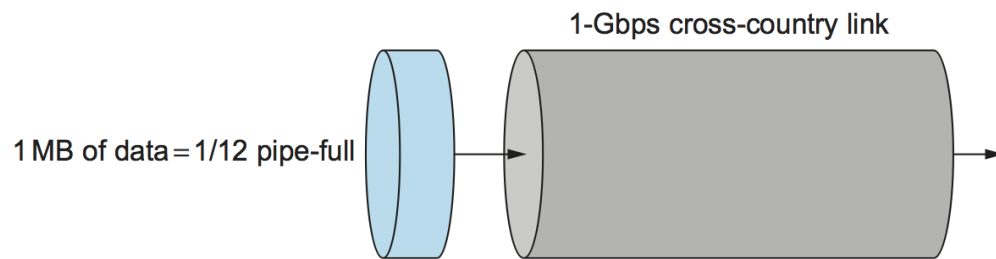


1-Gbps Network

Fits in 1/12th of an RTT

Delay $\times$ bandwidth product = 12.5 MB — far bigger than our 1 MB file.

The file behaves like a SINGLE PACKET.



A 1-MB file would fill the 1-Mbps link 80 times but only 1/12th of a 1-Gbps link." Analogy: a 1-MB file is to a 1-Gbps network what a 1-KB packet is to a 1-Mbps network.

Effective Throughput & Transfer Time

$$\text{Throughput} = \text{TransferSize} / \text{TransferTime}$$

$$\text{TransferTime} = \text{RTT} + (1/\text{Bandwidth}) \times \text{TransferSize}$$

Why Include RTT?

RTT accounts for the request message travelling to the server and the response data travelling back — the full round trip of a real request/response transfer, not just raw transmission.

As TransferSize $\rightarrow \infty$, effective throughput approaches the network's raw bandwidth. Enduring extra RTTs (e.g. retransmissions) hurts effective throughput most for small transfers.

Example: Effective Throughput

Problem: Fetch a 1 MB file across a 1-Gbps link with RTT = 100 ms. Find the effective throughput.

Step 1
Transmit time

$$1/(1 \text{ Gbps}) \times 1 \text{ MB} = 8 = 8 \times 10^6 \text{ bits} / 1 \times 10^9 = 8 \times 10^{-3} = 8 \text{ ms}$$

Step 2
Total transfer time

$$\text{TransferTime} = \text{RTT} + \text{Transmit} = 100 \text{ ms} + 8 \text{ ms} = 108 \text{ ms}$$

Step 3
Effective throughput

$$1 \text{ MB} / 108 \text{ ms} = 8 \times 10^6 / 0.108 \text{ s} = 7,40,74,074 \text{ bits} = 74.1 \text{ Mbps}, \text{ NOT the full 1 Gbps!}$$

The 1-Gbps "speed limit" is rarely reached for small transfers, RTT overhead dominates.

High-Speed Networks in the Real World

1

You upgrade your home internet from 100 Mbps to 1 Gbps but your video calls still stutter. Using today's concepts, brainstorm 3 possible explanations that are NOT about bandwidth.

2

Calculate: fetching a 500 KB web page over a 500-Mbps link with $RTT = 60$ ms. Find TransferTime and effective throughput. Compare with the raw bandwidth.

3

Discuss: Why do engineers designing satellite or undersea-cable networks worry MORE about RTT than about adding more bandwidth? Link your answer to the delay $\times$ bandwidth product.

Summary

Bandwidth

Capacity of the link (bits/sec); throughput is what's actually achieved in practice.

Latency

Time for data to travel = Propagation + Transmit + Queue delay.

Latency vs Bandwidth

Small transfers are latency-bound; large transfers are bandwidth-bound.

Delay $\times$ Bandwidth

The 'volume' of the pipe bits that can be in flight at once.

High-Speed Networks

Bandwidth keeps rising, but the speed of light and hence RTT stays fixed.

Effective Throughput

= $\text{TransferSize} / (\text{RTT} + \text{Size}/\text{Bandwidth})$; approaches raw bandwidth only for large transfers.